PROJECT REPORT · OCTOBER 2026 · BRANCH v2

NeighborHealth 2.0

Dengue early warning for Bengaluru, built on real surveillance data, with honest uncertainty

This report documents the rebuild of NeighborHealth from a first-semester prototype into a full-stack ML system: what it does, how every part works, what has and hasn't been tested yet, what was left out, and a prioritised list of what to do next to make it work end to end, and then to make it stand out.

Scope	8 steps: real data · forecasting ML · citizen reports · officials' tools · alerts · assistant · production engineering · real users
Code	~157 new files: pipeline/, ml/, apps/api/ (FastAPI), apps/web/ (Next.js), config/, infra/, docs/v2/
Status	Code complete. Data pipeline and ML tested (19 tests passing); API and web app written but not yet run
Next	Run it on your Mac, fix first-run errors, label 40 bulletins, publish the first real forecast

Contents

Section
1. What NeighborHealth 2.0 is
2. Why v1 had to be rebuilt
3. Architecture
4. The 8 steps, what was built
5. Frontend design
6. Tech stack
7. Repository map
8. How to run it
9. Testing status (honest)
10. What was left behind
11. To do now (prioritised)
12. Improvements to make it stand out
13. Resume bullets and interview talking points
14. Glossary

1. What NeighborHealth 2.0 is

Bengaluru reports thousands of dengue cases every year, mostly noticed after cases have already peaked. NeighborHealth turns the Karnataka health department's daily dengue bulletins into a weekly time series for BBMP, forecasts the next 1–4 weeks with calibrated ranges, and gives two groups of people tools to act early:

- **Residents:** a risk map by ward (with uncertainty), alerts for home, office or school in English, Kannada or Hindi, a way to report breeding spots with a photo, and a dengue assistant that only answers from official guidelines.
- **Health officials:** a dashboard of wards to prioritise, a queue of photo-checked citizen reports, field tasks with a full history, and optimised inspection routes for field teams.

Core principles

Principle	How it shows up in the code
Real data only	Synthetic v1 data is not used. Every number traces back to a government bulletin or NASA weather
Measure, don't assume	Extraction accuracy vs hand labels; forecasts vs baselines; interval coverage; explanation faithfulness
Honest uncertainty	50/80/95% ranges everywhere; ward values labelled as estimates; map fades uncertain wards
No invented reasons	Explanations come from SHAP values; AI text is fact-checked against them
Safety first	Assistant never diagnoses; warning signs trigger 'go to hospital / call 108'
Config, not code	City-specific settings in config/cities/bengaluru.yaml

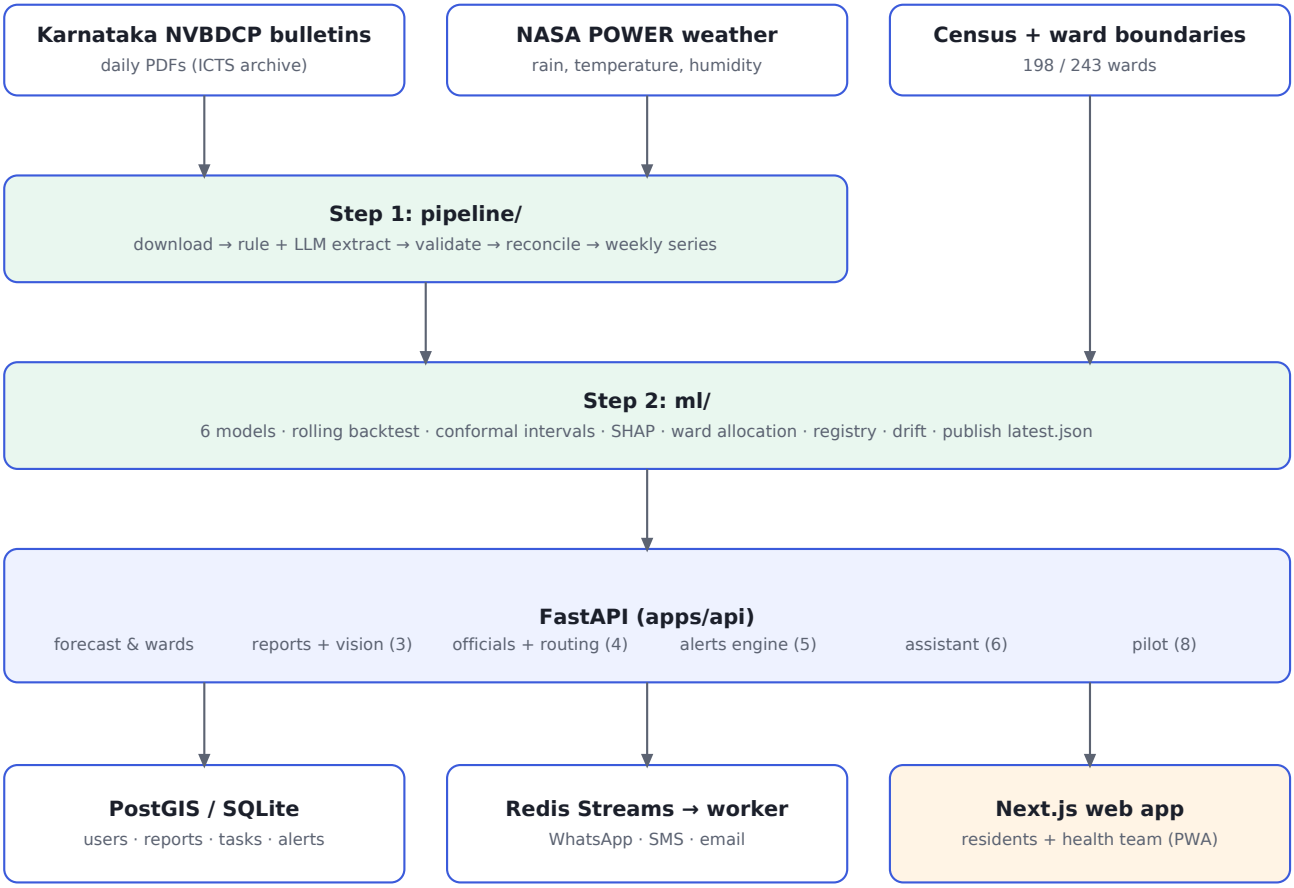
2. Why v1 had to be rebuilt

v1 (first semester) had a good structure (FastAPI, ward map, XGBoost, alerts, chat) but its machine learning wasn't real:

- **Circular labels:** case counts, community reports and the outbreak label were all generated by a hand-written formula of rain, temperature, humidity, month and density. The model was then trained on those same inputs, so it learned to copy the formula. The 0.986 ROC-AUC meant nothing about real dengue.
- **Test-set leakage:** early stopping and the decision threshold were tuned on the 2023–24 test set.
- **No real forecasting:** the 'forward 14-day' label actually used the same week's values.
- **Inconsistent claims:** README said AUC 0.96 and 'Parliamentary records'; metadata said 0.986; data was synthetic.
- **Auth bypassed** for testing in a commit, and a skin/cough 'health checker' making unvalidated medical judgments.

Interview story: "My first version had circular labels and leakage. I found it, threw the data away, and rebuilt on real government surveillance data with a pipeline whose accuracy I measured." Owning a mistake like this, with numbers, is a strong signal.

3. Architecture



Step 7 wraps everything: Docker · CI quality gate · weekly retrain · auth · metrics · config per city

The weekly loop

1. New bulletins are downloaded and extracted; the weekly BBMP case series and weather are rebuilt.
2. Every model is backtested week by week on all past data; the best is registered as a candidate.
3. The candidate replaces the champion only if it passes the quality gate and is at least as good on the same data.
4. The champion produces latest.json: forecasts, intervals, drivers, ward estimates, backtest scores and drift status.
5. The API serves it; the alerts engine checks every saved place and queues messages; the worker delivers them.
6. Verified citizen reports feed back into next week's ward allocation.

4. The 8 steps, what was built

Step 1: Real data

pipeline/

- Downloader for Karnataka NVBDCP daily dengue bulletins (ICTS archive): resumable, polite (1 req/s), manifest of every date, SSL fix + clear errors.
- Two independent extractors: a rule-based parser that rebuilds rows from word positions and picks the reading that passes arithmetic checks (IgM + NS1 = Total), and an LLM extractor (OpenRouter) with cached, reproducible results.
- Validator (row + bulletin checks), reconciler with provenance (which extractor, trusted, agreed, flags).
- Weekly series from year-to-date cumulative counts, so missing days don't lose cases; outlier removal; year-reset chaining; weekly testing volume.
- Weekly weather from raw NASA POWER JSON; data card; hand-labelling tool and accuracy report (docs/v2/extraction_eval.md).

Step 2: Forecasting ML

ml/

- Models: persistence, seasonal naive, SARIMA with Fourier seasonality, Poisson GLM, gradient boosting (LightGBM or scikit-learn), and an ensemble.
- Features built only from data known at the forecast origin (look-ahead test included): case lags and trend, same week last year, 4–8 week rain, temperature, humidity, testing volume and positivity, season.
- Rolling-origin backtest for horizons 1–4; online conformal 50/80/95% intervals that never use future residuals; MAE, WIS, coverage, skill vs baselines.
- SHAP explanations (LightGBM native TreeSHAP, exact linear SHAP for the GLM, permutation Shapley otherwise) grouped into readable factors.
- Ward allocation: 2023 ward polygons, population estimated from 2011 census points, density prior + verified-report adjustment, Monte Carlo uncertainty, data-driven risk bands.
- File model registry (+ optional MLflow), quality gate, champion/challenger promotion, rollback, PSI drift report, auto-generated model card.

Step 3: Citizen reports

apps/api/app/services/reports.py · hotspots.py · vision.py

- Photo + GPS intake with hard checks (inside city, real image, daily limit); EXIF stripped for privacy.
- Perceptual hash (dHash) to merge duplicates within 50 m / 14 days and to catch reused photos.
- Zero-shot CLIP photo check (optional) with an evaluation script for your own labelled photos.
- Reporter trust score (Beta posterior), spam rules, DBSCAN (haversine) hotspots weighted by trust.

Step 4: Officials' tools

apps/api/app/routers/officials.py · services/routing.py · tasks.py

- Overview: wards to prioritise, hotspots, queue size, task and alert stats.
- Review queue sorted by photo-check result; verifying a report creates a field task automatically.
- Task lifecycle (open → in progress → inspected → cleared → closed) with an audit trail.
- Route optimisation: OR-Tools vehicle routing with shift limits and priority-based dropping; heuristic fallback (sweep + nearest neighbour + 2-opt); reports time saved vs the naive order.

Step 5: Alerts

apps/api/app/services/alerts/

- Saved places (home/office/school) mapped to wards; alerts only on upward threshold crossings (hysteresis).
- De-duplication per place and forecast week, fatigue cap (2 per week), quiet hours in city time.
- Redis Streams outbox with consumer groups and crash recovery; in-memory queue for development.
- WhatsApp/SMS (Twilio REST) and email (SMTP), dry-run by default; templates in English, Kannada, Hindi; fatigue metrics and feedback.

Step 6: Assistant

apps/api/app/services/assistant/

- Corpus from WHO / NCVBDC / ICMR documents, page-aware chunks, BM25 retrieval (no external service).
- Answers only from retrieved passages with [n] citations, verified after generation; 'not in sources' otherwise.
- Never diagnoses; warning signs of severe dengue (three languages) trigger an urgent message, 108, and nearest hospitals (OpenStreetMap).
- Ward explanations written only from SHAP facts, checked by a verifier (numbers and causes); template fallback; evaluation script.
- The v1 skin/cough checker is removed from the v2 app.

Step 7: Production engineering

docker-compose.yml · infra/ · apps/api/app/main.py

- PostGIS schema (geometry, spatial indexes) with SQLite for zero-setup development; plain-SQL migrations.
- JWT auth with roles (citizen / official / admin), bcrypt, rate limits; v1's auth bypass is gone.
- Prometheus metrics, JSON logs with request IDs, health endpoint with forecast freshness, optional Sentry.
- Docker images for API and web, compose with Postgres, Redis, worker, web, MLflow; CI with tests, typecheck/build and the model quality gate; weekly retrain workflow.

Step 8: Real users

apps/api/app/services/pilot.py · docs/v2/PILOT_PLAYBOOK.md

- Group codes for RWAs/NSS (cohorts), consent records with versions, privacy-safe event tracking.
- Pilot metrics in the dashboard: residents, subscribers, weekly active users, reports, verification rate, tasks, assistant quality.
- Playbook: launch checklist, 7-week plan, outreach messages (RWA, NSS, BBMP officer), feedback survey, data deletion steps.

5. Frontend design

The web app was rebuilt from scratch in Next.js + TypeScript with a deliberate visual identity rather than a template.

Element	Choice and reason
Palette	Rain-paper off-white, monsoon ink #14213D, lake teal #0E7C7B for actions; risk ramp sage → marigold #F2B33D → #E06A2C → kumkum #9E2338, taken from Bengaluru's flower markets and readable for colour-blind users
Type	IBM Plex Sans + Plex Sans Devanagari + Noto Sans Kannada (one voice across 3 languages); Plex Condensed for numbers with tabular figures
Signature	Home page leads with the live forecast fan chart (50/80/95% bands) with rainfall bars hanging below on the same timeline, showing rain leading cases
Map	Full-bleed ward map; fill colour = estimated band, fill opacity = certainty; week selector; hotspots and reports layers; side panel / mobile bottom sheet
Ward panel	Rate per 100k with an 80% range bar, chance of high risk, next 4 weeks, SHAP drivers, 'explain in plain words', alert-me button
Pages	Home · Risk map · Forecast (with honest backtest table) · Report a spot · Alerts · Ask · Health team (5 tabs) · How it works · Sign in
Quality floor	Dark mode, 3 languages, keyboard focus, reduced motion, offline-capable PWA, mobile tab bar

6. Tech stack

Layer	Technology
Data pipeline	Python, pdfplumber, pandas, pydantic, OpenRouter (LLM extraction)
ML	scikit-learn, LightGBM, statsmodels, NumPy/SciPy; MLflow (optional)
API	FastAPI, PyJWT, bcrypt, psycopg 3, Redis, Prometheus client
Database	PostgreSQL + PostGIS (production), SQLite (development)
Geo / optimisation	Pure-NumPy geometry, DBSCAN (haversine), Google OR-Tools
Vision / NLP	CLIP zero-shot (transformers, optional), BM25 retrieval, LLM via OpenRouter
Web	Next.js 14, React 18, TypeScript, Tailwind, MapLibre (react-map-gl), Recharts, Radix UI, SWR
Messaging	Twilio (WhatsApp/SMS), SMTP email
Ops	Docker, docker-compose, GitHub Actions (CI + weekly retrain), service worker (PWA)

7. Repository map

```
neighborhealth-complete/            (branch v2)
├── README.md                       new v2 overview
├── config/cities/bengaluru.yaml    everything city-specific
├── config/guideline_sources.yaml   official documents for the assistant
├── pipeline/                       Step 1: download, extract (rule + LLM), validate, build, eval
├── ml/                             Step 2: features, models, backtest, explain, wards, registry, publish
├── apps/api/                       FastAPI app (Steps 3-8)
│   ├── app/routers/                auth, public, reports, officials, alerts, assistant
│   ├── app/services/               reports, vision, hotspots, tasks, routing, alerts/, assistant/, clinics, pil
│   ├── app/migrations/             SQL schema (common + PostGIS)
│   ├── scripts/                    eval_vision.py, eval_explanations.py
│   └── tests/
├── apps/web/                       Next.js app (all pages + components)
├── infra/                          Makefile + GitHub workflows (move into place once)
├── docker-compose.yml
├── docs/v2/                        STEP1_DATA, DATA_CARD, PILOT_PLAYBOOK (+ generated reports)
└── backend/, frontend/             v1 code (kept for reference; not used by v2)
```

8. How to run it

```
cd ~/Desktop/projects/Hackathon/neighborhealth-complete
git checkout v2
cp infra/nh.mk Makefile                       # one-time
mkdir -p .github/workflows && cp infra/github-workflows/*.yml .github/workflows/
python3 -m venv .venv && source .venv/bin/activate
make setup                    # Python + npm packages, creates .env files
                              # add OPENROUTER_API_KEY to apps/api/.env
make data                     # Step 1: real bulletins + weather -> weekly series
python -m pipeline status     # how many PDFs per year?
python -m pipeline label --n 40 && python -m pipeline eval
make weekly                   # backtest, promote, publish forecast, run alerts
make guidelines clinics       # assistant sources + nearby facilities
make officials EMAIL=you@example.com PASSWORD=choose-a-password
make api                      # terminal 1 -> http://localhost:8000/docs
make worker                   # terminal 2 (alerts, dry-run by default)
make web                      # terminal 3 -> http://localhost:3000
make test                     # all Python tests
```

Or with Docker: `docker compose up --build`, then `docker compose run --rm api python -m ml weekly`.

9. Testing status (honest)

Part	Status	Notes
Pipeline: parser, validator, reconcile, weekly series	Tested: 10/10 pass	On sample PDFs that copy real 2024 bulletin rows; not yet on the real archive
ML: features, models, backtest, intervals, SHAP, wards	Tested: 9/9 pass	On synthetic data in tests only (never used for results)
Downloader	Fixed, not re-run	Your first run produced no files (likely SSL); now fails with a clear message
Weekly MLOps flow end to end	Not run	Interrupted before the full run on real ward files
API (FastAPI routes, DB, auth)	Written, never run	Packages weren't installable in the build environment; expect first-run fixes
API service unit tests	Written, not run	apps/api/tests/test_services.py
Web app (Next.js)	Written, never built	Expect some TypeScript/import fixes on first npm run build
Docker / CI workflows	Written, not run	CI runs on first push of the v2 branch
Kannada / Hindi text	Needs review	Written carefully but must be checked by native speakers

10. What was left behind

These were in the 2.0 plan but were simplified or not built. None blocks the first working version.

Data

- IDSP/NCDC weekly outbreak reports are not ingested (only Karnataka daily bulletins).
- No fallback yet if many bulletins lack BBMP's cumulative column (build weekly counts from 'on the day' numbers).
- Archive coverage (which years exist) is unknown until the download runs.

ML

- The spatio-temporal graph model (ward adjacency) stretch goal is not built.
- Ward population uses an inverse-distance approximation from 2011 ward points, not a true polygon overlay (no 2011 polygons).
- Reporting delays and testing volume are features, not modelled explicitly (no nowcasting).
- No probabilistic outbreak alert (probability that cases exceed a threshold).

Reports & vision

- Photo check is zero-shot only; no labelled photo set, no fine-tuned model, thresholds not tuned.
- No automatic face / number-plate blurring in uploaded photos.

Alerts

- WhatsApp via Twilio needs pre-approved message templates for messages sent outside a 24-hour window; not set up.
- Inbound 'STOP' replies (Twilio webhook) are not handled; unsubscribing works in the app only.

Assistant

- Retrieval is keyword (BM25) only; no embeddings, reranker or multilingual retrieval.
- Guideline URLs may have moved; no Q&A; evaluation set yet.
- Non-English ward explanations aren't number-checked as strictly as English ones.

Platform

- PostGIS columns exist, but most spatial queries still use the Python fallback.
- Rate limiting is in-process (not shared across instances); no migration rollbacks.
- No password reset, email verification, or admin screen for roles (CLI only).
- Token stored in localStorage (simpler, but httpOnly cookies are safer against XSS).
- No data-deletion endpoint (manual SQL in the playbook); no privacy policy / terms page.
- No integration tests (FastAPI TestClient), frontend tests (Playwright), or load tests.
- No Grafana dashboards provisioned; no deployment target (Render/Fly/Vercel, domain, HTTPS).
- Offline report submission queue (submit later when back online) is not built.
- Many UI strings on the forms and the health-team page are English-only.

11. To do now (prioritised)

Work top to bottom. Each block unlocks the next one.

A. Make the data real (this week)

☐ Run the one-time infra copy, make setup, then make data. If the download fails, send the exact error.

☐ python -m pipeline status: note how many PDFs per year exist.

☐ Open 3–4 real PDFs and compare with extracted_rule.jsonl; send parser failures so the parser can be fixed on real layouts.

☐ Hand-label 40 bulletins (python -m pipeline label), run eval, and set build --prefer to the better extractor.

☐ If BBMP's cumulative column is often missing, add the 'on the day' fallback for the weekly series.

B. First real forecast

☐ make weekly; read docs/v2/MODEL_CARD.md.

☐ If no model beats the seasonal-naive baseline, that's a real finding: report it honestly, then improve features/data.

☐ Check interval coverage is close to 50/80/95%. python -m ml gate must pass before anything is shown to users.

C. Bring up the app

☐ make api: fix first-run errors; open /docs and try each endpoint.

☐ npm run typecheck and npm run build in apps/web; fix TypeScript errors; check every page on phone width and in dark mode.

☐ Run make test; get all Python tests green.

☐ Create an official account and walk through: report → review → task → route → cleared.

D. Make the assistant and alerts safe

☐ make guidelines: fix any broken URLs; ask 20 questions and check every citation by hand.

☐ Run alerts in dry-run, read data/v2/outbox.log, then test on your own phone with Twilio (WhatsApp sandbox first).

☐ Get Kannada and Hindi text reviewed by native speakers.

E. Ship and pilot

☐ Commit the v2 branch, push, open a PR, let CI run.

☐ Deploy: API + Postgres on Render/Fly/Railway, web on Vercel, a custom domain with HTTPS.

☐ Write a short privacy notice and terms page.

☐ Start the pilot with 1 RWA or your NSS unit using the playbook; collect the metrics.

12. Improvements to make it stand out

Horizon	Improvement	Why it matters
Short	Probabilistic outbreak alerts: P(next week > seasonal 75th percentile), scored with Brier score	Decision-ready output for officials, a new metric
Short	Embeddings + reranker for the assistant; a 50-question eval set with citation accuracy	RAG evaluation is a core AI-engineering skill
Short	Playwright end-to-end tests + FastAPI integration tests in CI	Production-grade engineering signal
Short	Demo video, README GIF, architecture diagram	Recruiters judge in 30 seconds
Medium	All Karnataka districts + a hierarchical model (districts share seasonality)	More data, better accuracy, harder ML
Medium	Spatio-temporal graph model over neighbouring districts/wards	The original stretch goal; strong ML story
Medium	Nowcasting for reporting delays; testing-adjusted target	Shows epidemiological maturity
Medium	Labelled photo set (200+) and a fine-tuned small vision model vs zero-shot	Real computer-vision result with numbers
Medium	Move spatial queries to PostGIS; Redis rate limiting; httpOnly cookie auth; Grafana dashboards	Scale and security talking points
Long	Generalise to other cities (Chennai, Mumbai) using the city config	Proves the design is reusable
Long	Write a short technical report / arXiv preprint on the extraction + forecasting pipeline	Rare and impressive for a student
Long	Partner with a BBMP ward health office or a public-health department for real use	Real impact beats any feature

13. Resume bullets and interview talking points

Resume bullets (replace the blanks with your real numbers after running)

```
NeighborHealth: dengue early-warning platform for Bengaluru              [live] [github]
• Built a document-AI pipeline turning ___ daily government dengue bulletins (PDF) into a
  validated weekly series; rule-based parser ___% vs LLM ___% field accuracy on ___
  hand-labelled bulletins.
• Forecast weekly BBMP cases 1-4 weeks ahead with rolling backtests and conformal intervals;
  champion model beat the seasonal baseline by ___% MAE with ___% 80%-interval coverage;
  SHAP explanations with a fact-checking verifier (___% faithful).
• Shipped FastAPI + PostGIS + Next.js platform: photo-checked citizen reports (dedupe, trust
  scores, DBSCAN hotspots), OR-Tools inspection routing (___% less travel time), and
  multilingual WhatsApp/SMS alerts via Redis Streams with quiet hours and fatigue caps.
• MLOps: weekly retrain -> backtest -> quality-gated promotion, drift monitoring, CI;
  piloted with ___ residents in ___ RWAs (___ verified reports).
```

Questions this project will trigger, and what to cover

Question	What to cover
Why did you rebuild it?	Circular labels + test leakage in v1; how you found it; what 'real data' meant.
How accurate is the PDF extraction?	Two extractors, validator, gold labels, accuracy table, trusted vs flagged rows.
How do you know the forecast is good?	Rolling-origin backtest, baselines, WIS, interval coverage, quality gate.
Why conformal intervals?	Distribution-free, calibrated from real past errors, no leakage (only observed residuals).
How do you get ward risk without ward data?	Allocation model, assumptions, Monte Carlo uncertainty, and saying it's an estimate.
How do you stop the LLM from making things up?	Retrieval-only answers, citation checks, SHAP-only explanations with a verifier, fallbacks.
How would this scale to all of India?	District-level ingestion, partitioning, hierarchical models, queue-based alerts, PostGIS.
What would you do differently?	Be honest: e.g. label data earlier, test the API sooner, start the pilot earlier.

14. Glossary

Term	Meaning
BBMP	Bruhat Bengaluru Mahanagara Palike, Bengaluru's municipal corporation
NVBDCP / NCVBDC	India's national vector-borne disease programme (state unit publishes the bulletins)
Seasonal naive	Baseline forecast: same week last year
Rolling-origin backtest	Re-run the forecast each past week using only data known then
Conformal interval	A range built from past forecast errors that covers the truth a stated % of the time
WIS	Weighted interval score: scores the whole forecast range; lower is better
SHAP	Method that splits a prediction into contributions from each input
PSI	Population stability index: measures how much recent data differs from training data
DBSCAN	Clustering that finds dense groups of points (hotspots)
Hysteresis	Alert only when risk crosses the threshold upward; re-arm after it drops
PWA	Progressive web app: installable on a phone, works offline